TrackLib

Référence des fonctions et méthodes par objet

Dépôt : github.com/umrlastig/tracklib (LASTIG, IGN)
Documentation source : tracklib.readthedocs.io (version 0.8)
Document généré le 18 octobre 2026

Contenu

1. Obs (observation)
2. GeoCoords, ENUCoords, ECEFCoords et fonctions de coordonnées
3. Bbox (boîte englobante)
4. ObsTime (horodatage)
5. Track (trace GPS)
6. TrackCollection (ensemble de traces)
7. Simplification (algo.simplification)
8. Comparaison, appariement et fusion (algo.comparison)
9. Calage et map-matching (algo.mapping)
10. Fonctions géométriques (util.geometry)
11. Topology (util.topology)
12. Entrées / sorties (io)
13. Points d'attention relevés dans le code
14. Ce que ce document ne couvre pas

Méthode : ce document a été établi à partir du code source publié dans la documentation en ligne (pages 'Source code for ...') et des pages d'API. GitHub bloquant l'accès automatisé, le dépôt lui-même n'a pas été parcouru. Beaucoup de docstrings du projet valent encore 'TODO' : lorsque le rôle d'une méthode est déduit de son nom et de son code, la description le reflète sans citer de documentation officielle. Aucun exemple n'a été exécuté.

1. Obs (module core.obs)

Une observation est un point d'une trace : une position et un instant. Une Track est essentiellement une liste d'objets Obs. La classe est volontairement légère : les calculs géométriques sont délégués à l'objet de coordonnées.

Constructeur

Fonction / méthode	Rôle
`Obs(position, timestamp=None)`	position : un objet ENUCoords, GeoCoords ou ECEFCoords. timestamp : un ObsTime ; s'il est omis, ObsTime() est utilisé (1er janvier 1970, 00:00:00).

Attributs

Fonction / méthode	Rôle
`position`	Coordonnées du point.
`timestamp`	Instant d'acquisition (ObsTime).
`features`	Liste des valeurs des analytical features (AF) du point, indexée par numéro. C'est la Track qui fait le lien entre nom et numéro.
`gdop, pdop, vdop, hdop, tdop`	Dilutions de précision GNSS (géométrique, position, verticale, horizontale, temporelle). 0 par défaut.
`nb_sats`	Nombre de satellites. 0 par défaut.
`mask, code`	Champs GNSS bruts, non documentés. 0 par défaut.
`azimut, elevation`	Champs libres initialisés à 0. Ne sont pas calculés automatiquement.

Méthodes

Fonction / méthode	Rôle
`copy()`	Copie profonde de l'observation.
`distanceTo(obs)`	Distance 3D vers une autre observation (délègue à position.distanceTo). Les deux positions doivent être du même type.
`distance2DTo(obs)`	Distance 2D vers une autre observation. Refuse les coordonnées ECEF.
`azimuthTo(obs)`	Azimut de la direction vers l'autre observation. Même type de coordonnées requis.
`elevationTo(obs)`	Angle d'élévation vers l'autre observation. Même type de coordonnées requis.
`__str__()`	Texte 'timestamp position'.
`obs[i]`	Lit la i-ème feature analytique du point (accès par numéro).
`obs[i] = valeur`	Modifie la i-ème feature analytique du point.

2. Coordonnées (module core.obs_coords)

Trois classes de coordonnées : GeoCoords (géographiques), ENUCoords (projection locale East-North-Up, en mètres) et ECEFCoords (géocentriques). Les trois exposent les accesseurs getX/getY/getZ et setX/setY/setZ, ce qui permet de les manipuler de façon uniforme.

GeoCoords(lon, lat, hgt=0.0)

lon et lat en degrés décimaux, hgt en mètres. Attention à l'ordre : longitude en premier.

Fonction / méthode	Rôle
`copy()`	Copie profonde.
`__str__()`	Représentation '[lon=..., lat=..., hgt=...]'.
`toECEFCoords()`	Convertit en coordonnées ECEF absolues (ellipsoïde WGS84).
`toENUCoords(base)`	Convertit en ENU local par rapport à base (GeoCoords ou ECEFCoords). Si base est un entier, appelle toProjCoords(base).
`toGeoCoords()`	Renvoie une copie (permet de garantir le type).
`toProjCoords(srid)`	Projection vers un SRID. Seul 2154 (Lambert 93) est implémenté ; tout autre code provoque un message d'erreur puis exit().
`distanceTo(point)`	Distance 3D (calculée en passant par l'ECEF).

Fonction / méthode	Rôle
`distance2DTo(point)`	Distance 2D (norme planimétrique après conversion ENU).
`elevationTo(point)`	Élévation vers un autre point, en radians.
`azimuthTo(point)`	Azimut vers un autre point, en radians (atan2(E, N)).
`getX/getY/getZ`	Renvoient lon, lat, hgt.
`setX/setY/setZ`	Modifient lon, lat, hgt.
`plot(sym='ro')`	Trace le point avec matplotlib.

ENUCoords(E, N, U=0)

Coordonnées locales en mètres. C'est la seule des trois classes à fournir des opérations vectorielles et des transformations affines.

Fonction / méthode	Rôle
`__str__(), __repr__()`	Représentation '[E=..., N=..., U=...]'.
`__eq__(other)`	Égalité avec tolérance de 1e-4 sur chaque composante.
`copy()`	Copie profonde.
`toECEFCoords(base)`	Convertit en ECEF absolu à partir de la base.
`toGeoCoords(base)`	Convertit en géographique. Si base est un entier : 2154 (Lambert 93) ou 32601 à 32760 (UTM nord/sud).
`toENUCoords(base1, base2)`	Passe d'un ENU relatif à base1 à un ENU relatif à base2.
`norm2D()`	Norme euclidienne planimétrique sqrt(E2 + N2).
`norm()`	Norme euclidienne 3D.
`dot(point)`	Produit scalaire avec un autre vecteur.
`elevationTo(point)`	Élévation vers un autre point, en radians.
`azimuthTo(point)`	Azimut vers un autre point, en radians.
`__sub__(p)`	Différence vectorielle self - p (opérateur -).
`__add__(p)`	Somme vectorielle (opérateur +).
`distance2DTo(point)`	Distance planimétrique.
`distanceTo(point)`	Distance 3D.
`rotate(theta)`	Rotation 2D autour de l'origine (0, 0), theta en radians. Modifie le point en place.
`scale(h)`	Homothétie 2D de rapport h centrée sur l'origine. Multiplie E et N ; U n'est PAS modifié.
`translate(tx, ty, tz=0)`	Translation 3D en place.
`getX/getY/getZ, setX/setY/setZ`	Accès à E, N, U.
`plot(sym='ro')`	Trace le point avec matplotlib.

ECEFCoords(X, Y, Z)

Fonction / méthode	Rôle
`__str__(), copy()`	Représentation texte ; copie profonde.
`toGeoCoords()`	Convertit en longitude, latitude, hauteur.
`toENUCoords(base)`	Convertit en ENU local par rapport à base.
`toECEFCoords()`	Renvoie une copie (garantit le type).
`elevationTo(point), azimuthTo(point)`	Élévation et azimut vers un autre point, en radians.
`dot(point), norm()`	Produit scalaire ; norme 3D.
`scalar(factor)`	Multiplie le vecteur par un facteur, en place.
`__sub__(p), __add__(p)`	Différence et somme vectorielles.
`distanceTo(point)`	Distance 3D en mètres.
`getX/getY/getZ, setX/setY/setZ`	Accès à X, Y, Z.

Fonctions et constantes du module

Fonction / méthode	Rôle
`makeCoords(x, y, z, srid)`	Fabrique un objet de coordonnées à partir de trois nombres et d'un nom de système ('ENU', 'GEO' ou 'ECEF'). Autre valeur : erreur puis exit().
`Re, Be, Fe, Ee`	Constantes de l'ellipsoïde : demi-grand axe 6378137,0 ; demi-petit axe ; aplatissement ; excentricité.
`STANDARD_PROJ`	Choix de la projection Geo vers ENU : 1 = plane (par défaut), 2 = stéréographique.
`_proj(coords, srid), _unproj(coords, srid)`	Fonctions internes de projection. Direct : 2154. Inverse : 2154 et UTM (32601-32799).

3. Bbox (module core.bbox)

Boîte englobante définie par son coin inférieur gauche (ll) et supérieur droit (ur). Bbox(ll, ur) mémorise aussi le système de coordonnées dans l'attribut srid.

Fonction / méthode	Rôle
`copy(), __str__()`	Copie profonde ; description des deux coins.
`getSrid()`	'ENU', 'GEO' ou 'ECEF'.
`getLowerLeft(), getUpperRight()`	Renvoient les deux coins.
`getXmin(), getYmin(), getXmax(), getYmax()`	Bornes de la boîte.
`getDx(), getDy(), getDimensions()`	Largeur, hauteur, ou tuple (Dx, Dy).
`setXmin(), setYmin(), setXmax(), setYmax()`	Modifient les bornes.
`contains(point)`	Teste si un point est dans la boîte (via le polygone).
`geom()`	Convertit la boîte en Polygon (util).
`addMargin(margin=0.05)`	Agrandit la boîte d'une marge relative à ses dimensions.
`translate(dx, dy)`	Translation 2D de la boîte.
`rotate(theta)`	Rotation 2D des deux coins (en radians).
`scale(h)`	Homothétie 2D des deux coins.
`toECEFCoords(base), toGeoCoords(base), toENUCoords(base)`	Conversion de coordonnées appliquée aux deux coins.
`bbox1 + bbox2`	Union : boîte englobant les deux boîtes.
`bbox1 & bbox2`	Intersection : déclarée mais non implémentée (renvoie None).
`bbox[i], bbox[i] = v`	Accès par indice 0 à 3 ou par nom 'xmin', 'xmax', 'ymin', 'ymax'.
`asTuple()`	Tuple (xmin, xmax, ymin, ymax).
`plot(sym='b-', append=...)`	Dessine le rectangle avec matplotlib.

Dans le code, la classe s'appelle Bbox (et non BBox). Les méthodes toXxxCoords appliquent la conversion à ll et ur, sans renvoyer de nouvelle boîte.

4. ObsTime (module core.obs_time)

Horodatage d'une observation. ObsTime(year=1970, month=1, day=1, hour=0, min=0, sec=0, ms=0, zone=0). Le premier argument peut aussi être une chaîne, lue selon le format de lecture courant. Le format de lecture par défaut est '2D/2M/4Y 2h:2m:2s'. Les codes de format sont 1D/2D, 1M/2M, 2Y/4Y, 1h/2h, 1m/2m, 1s/2s et 1z/2z/3z (fractions de seconde).

Méthodes d'instance

Fonction / méthode	Rôle
`copy()`	Copie profonde.
`toAbsTime()`	Nombre de secondes écoulées depuis le 01/01/1970 (flottant). Ignore les secondes intercalaires.

Fonction / méthode	Rôle
`addSec(nb), addMin(nb), addHour(nb), addDay(nb)`	Renvoient un nouvel ObsTime décalé de nb secondes, minutes, heures ou jours.
`getDayOfWeek()`	Jour de la semaine ('Mon' à 'Sun').
`printZone()`	Code de fuseau ISO 8601 ('Z' ou '+01:00').
`timeWithZone()`	Date au format GPX avec fuseau.
`convertToZone(zone)`	Convertit vers un autre fuseau horaire (décalage en heures).
`__str__()`	Texte selon le format d'affichage courant.
`t1 - t2`	Différence en secondes (flottant).
`==, !=, <, >, <=, >=`	Comparaisons chronologiques. L'égalité compare année, mois, jour, heure, minute, seconde et ms ; le fuseau est ignoré.

Méthodes statiques

Fonction / méthode	Rôle
`ObsTime.readTimestamp(texte)`	Construit un ObsTime à partir d'une chaîne selon le format de lecture.
`ObsTime.readUnixTime(secondes)`	Construit un ObsTime à partir de secondes écoulées depuis 1970.
`ObsTime.now()`	Date et heure courantes (millisecondes à 0).
`ObsTime.random()`	Date aléatoire entre le 01/01/1970 et le 01/01/2050.
`ObsTime.isLeapYear(year)`	Teste si une année est bissextile.
`ObsTime.setPrintFormat(fmt), getPrintFormat()`	Définit / lit le format d'affichage (global à la classe).
`ObsTime.setReadFormat(fmt), getReadFormat()`	Définit / lit le format de lecture (global à la classe).

Constantes : BASE_YEAR = 2000 (pour le code d'année sur 2 chiffres), UNIX_BASE_YEAR = 1970, ROUND_TO_SEC = 1.

5. Track (module core.track)

Représentation d'une trace GPS. Track(list_of_obs=None, user_id=0, track_id=0, base=None). Attributs : uid, tid, base (point de base ECEF/Geo de la projection ENU) et no_data_value. La trace garde aussi un dictionnaire de ses analytical features (AF) : des champs calculés ou importés, un par point (vitesse, abscisse curviligne, etc.). Les noms 'x', 'y', 'z', 't', 'timestamp' et 'idx' sont des AF virtuelles toujours disponibles et ne peuvent pas être réutilisés comme noms.

5.1 Informations générales

Fonction / méthode	Rôle
`copy()`	Copie profonde de la trace.
`size(), len(track)`	Nombre de points.
`getSRID()`	Type de coordonnées du premier point : 'Geo', 'ENU' ou 'ECEF'.
`length()`	Longueur totale : somme des distances entre points consécutifs.
`duration()`	Durée en secondes entre le premier et le dernier point.
`nanduration(no_data_value=None)`	Durée en ignorant les horodatages à la valeur par défaut (1970-01-01 00:00:00).
`frequency(mode)`	Fréquence moyenne : Hz en mode 'temporal', points par mètre en mode 'spatial'.
`interval(mode)`	Inverse de la fréquence.
`summary()`	Affiche un résumé : numéro, utilisateur, nombre de points, système, dates, durée, longueur, AF.
`print(n=-1, af_names=...)`	Affiche les n premiers points avec leurs AF.
`toWKT()`	Chaîne WKT LINESTRING en X Y (lon/lat en Geo, E/N en ENU).
`count_missing_geom(no_data_value)`	Compte les points dont une coordonnée vaut la valeur 'sans donnée'.
`count_missing_temps()`	Compte les points dont l'horodatage vaut 0 (date par défaut).
`count_af_value(af_name, val)`	Compte les points où une AF vaut val.

5.2 Systèmes de coordonnées et fuseaux horaires

Fonction / méthode	Rôle
`toECEFCoords(base=None)`	Convertit la trace en ECEF. Depuis ENU, une base est nécessaire.
`toENUCoords(base=None)`	Convertit en ENU. Sans base, le premier point sert de base et un avertissement s'affiche. Depuis ENU : changement de base.
`toGeoCoords(base=None)`	Convertit en coordonnées géographiques.
`toProjCoords(srid)`	Projette une trace Geo vers un SRID (2154 seulement).
`toENUCoordsIfNeeded()`	Convertit en ENU seulement si la trace est en Geo. Renvoie la base utilisée, ou None.
`toImageCoords(P1, P2, p1, p2)`	Convertit en coordonnées d'image à partir de deux points de calage.
`getTimeZone()`, `setTimeZone(zone)`	Lit / force le fuseau des horodatages.
`convertToTimeZone(zone)`	Convertit les horodatages vers un autre fuseau.

5.3 Accès aux observations et aux valeurs

Fonction / méthode	Rôle
`getFirstObs(), getLastObs()`	Première et dernière observation.
`getObs(i), getObsList()`	Une observation ; la liste complète (non copiée).
`getCoord(i=None)`	Position du point i, ou liste de toutes les positions.
`getX(i), getY(i), getZ(i)`	Coordonnée d'un point, ou liste complète si i est omis.
`getT(i)`	Temps absolu en secondes (point ou liste).
`getTimestamps(i)`, `getTimestamps_str(i)`	Horodatages en ObsTime ou en texte.
`getCentroid()`	Position moyenne de la trace.
`getNearestObs(o)`, `getFurthestObs(o)`	Copie du point le plus proche / le plus éloigné d'une observation donnée.
`getMedianObs()`	Observation au temps médian (échantillonnée sur la trace).
`getMedianObsInTime()`	Observation à mi-durée de la trace.
`getEnclosedPolygon()`	Polygone formé par la trace.
`getMinX/Y/Z(), getMaxX/Y/Z()`	Extrema des coordonnées.
`getLowerLeftPoint()`, `getUpperRightPoint()`	Coins de la boîte englobante.
`bbox()`	Boîte englobante (Bbox).

5.4 Modification de la liste de points

Fonction / méthode	Rôle
`addObs(obs)`	Ajoute un point en fin de trace.
`insertObs(obs, i=None)`	Insère à l'indice i, ou à la bonne place chronologique si i est omis.
`insertObsInChronoOrder(obs)`	Insère selon l'horodatage.
`setObs(i, obs)`, `setObsList(liste)`	Remplace un point ; remplace toute la liste.
`removeObs(arg)`, `removeFirstObs()`, `removeLastObs()`	Supprime un point par indice ou par horodatage ; le premier ; le dernier.
`removeObsList(liste)`	Supprime plusieurs points (indices entiers ou ObsTime).
`popObs(idx)`	Supprime et renvoie un point.
`sort()`	Trie les points par horodatage.
`sortRadix()`	Tri par horodatage en O(n).
`isSorted()`	Teste si les horodatages sont strictement croissants.
`reverse()`	Renvoie une copie dont l'ordre des points est inversé.
`setOrder(name, start)`	Crée une AF 'order' numérotant les points. (Le paramètre name n'est pas utilisé.)

Fonction / méthode	Rôle
`setUid(id), setTid(id)`	Définit l'identifiant de l'utilisateur / de la trace.

5.5 Nettoyage

Fonction / méthode	Rôle
`removeNoDataValues(no_data_value)`	Supprime les points ayant une coordonnée égale à la valeur 'sans donnée'.
`removeTpsDup(et=1e-3)`	Corrige les points ayant le même horodatage (à 1 ms près) mais des positions différentes en réinterpolant les temps. Renvoie une nouvelle trace.
`removePosDup(ep=1e-2)`	Supprime en place les points à la même position (à 1 cm près) et d'horodatages différents.
`removeObsDup(et, ep)`	Prévue pour supprimer les doublons exacts ; dans le code actuel la fonction est vide (renvoie None).
`cleanDuplicates(code='XYZ')`	Supprime les points identiques au précédent selon des critères : X, Y, Z, T, AF (ex. 'XYT').
`removeIdleEnds(parameter, mode='begin')`	Supprime les points immobiles au début ('begin') ou à la fin ('end'). Renvoie une nouvelle trace.
`digitize(r, clean=True)`	Arrondit les coordonnées à la résolution r ; supprime ensuite les doublons. Renvoie une copie.
`makeOdd(), makeEven()`	Retire le dernier point si besoin pour obtenir un nombre impair / pair de points.
`loop(add=False)`	Ferme la trace : ajoute une copie du premier point, ou aligne le premier point sur le dernier.
`dual()`	Renvoie une trace de n-1 points placés au milieu de chaque segment (position et temps).

5.6 Extraction et temps

Fonction / méthode	Rôle
`extract(id_ini, id_fin)`	Sous-trace entre deux indices inclus (les points ne sont pas copiés).
`extractSpanTime(tini, tfin=None)`	Sous-trace entre deux instants (points copiés). Accepte aussi une autre trace comme intervalle.
`addSeconds(sec_number)`	Ajoute des secondes (éventuellement négatives) à tous les horodatages.
`incrementTime(dt=1, offset=0)`	Ajoute i*dt + offset secondes au i-ème point. Utile pour des traces sans horodatage valide.
`roundTimestamps(unit)`	Arrondit les horodatages.

5.7 Analytical features (AF)

Fonction / méthode	Rôle
`hasAnalyticalFeature(name)`	Teste l'existence d'une AF (y compris les AF virtuelles x, y, z, t, timestamp, idx).
`getListAnalyticalFeatures()`	Liste des noms d'AF de la trace.
`getAnalyticalFeature(name), getAnalyticalFeatures(names)`	Valeurs d'une AF ou de plusieurs AF, pour tous les points.
`getObsAnalyticalFeature(name, i), getObsAnalyticalFeatures(names, i)`	Valeur(s) d'AF pour le point i.
`setObsAnalyticalFeature(name, i, val)`	Modifie la valeur d'une AF pour le point i.
`createAnalyticalFeature(name, val_init=0.0)`	Crée une AF, initialisée à une valeur ou à une liste. Avertissement si elle existe déjà.
`addAnalyticalFeature(algorithm, name=None)`	Crée et calcule une AF en appliquant une fonction algorithm(track, i) à chaque point.
`updateAnalyticalFeature(name, new_val)`	Remplace les valeurs d'une AF existante.
`removeAnalyticalFeature(name)`	Supprime une AF.
`fillAnalyticalFeature(af_name, X, T, tol=1, agg=moyenne)`	Remplit une AF à partir de mesures externes horodatées, en agrégeant celles proches (tolérance en secondes).
`setX/Y/Z/TFromAnalyticalFeature(af_name)`	Remplace la coordonnée (ou le temps) par les valeurs d'une AF.
`setX/Y/ZFromFunction(function)`	Remplace la coordonnée par function(track, i).

Fonction / méthode	Rôle
`isAFTransition(af_name)`	Teste si l'AF est un marqueur binaire (0/1) de changement de régime.
`estimate_speed(kernel=None)`	Calcule la vitesse de chaque point (différences finies, ou lissée si un noyau est donné).
`getSpeed(), getAbsCurv()`	Renvoient les AF de vitesse et d'abscisse curviligne ; erreur si elles n'ont pas été calculées.

5.8 Calculs algébriques sur les AF

Fonction / méthode	Rôle
`operate(operator, arg1, arg2, arg3), op(...)`	Calculs sur les AF. Soit avec un opérateur de la classe Operator, soit avec une expression algébrique : track.operate('v=3.6*D{s}/D{t}'). Opérateurs : + - / * ^ ; D = dérivée, I = intégrale, D2 = dérivée seconde, >> et << = avance/retard. Le côté gauche d'un '=' crée une AF.
`bioperate(track, expression), biop(...)`	Même chose entre deux traces ; les AF de la seconde trace sont suffixées par le caractère degré.
`query(cmd)`	Requête de type SQL sur les points : SELECT * / SELECT champs / SELECT AGG(champ), avec WHERE (conditions séparées par AND/OR, pas de parenthèses). Agrégateurs : SUM, AVG, COUNT, VAR, MEDIAN, ARGMIN, ARGMAX, MIN, MAX, RMSE, MAD, STDDEV, ZEROS.

5.9 Traitements du signal et bruit

Fonction / méthode	Rôle
`resample(delta, algo=1, mode=1, npts=None, factor=1)`	Rééchantillonne la trace (ENU requis). mode : 1 spatial, 2 temporel. algo : 1 linéaire, 2 plaques minces, 3 B-splines, 4 processus gaussien. En mode temporel, delta peut être un intervalle, une liste d'horodatages ou une trace de référence. Les AF existantes sont effacées.
`smooth(width=1)`	Lissage gaussien. Voir la remarque en section 13.
`noise(sigma=5, kernel=None, force=False, cycle=False, control=[], n=1)`	Renvoie une trace bruitée (bruit corrélé selon le noyau). Sans noyau : bruit blanc. Doit être réaffecté : t = t.noise(...).
`mapOn(reference, TP1, TP2, ...)`	Alignement affine sur une autre trace par moindres carrés (marqué obsolète dans le code).

5.10 Transformations géométriques

Fonction / méthode	Rôle
`translate(tx, ty, tz=0)`	Translation (ENU).
`rotate(theta, center=None)`	Rotation 2D (ENU), en radians, autour de center ou de l'origine.
`rotate3D(R)`	Rotation 3D par une matrice 3x3 (ENU ou ECEF).
`scale(h)`	Homothétie 2D de rapport h centrée sur l'origine (ENU). Modifie la trace en place.
`scale3D(h, center=None)`	Homothétie 3D autour de center (le centroïde par défaut), ENU ou ECEF.
`symmetrize(dim, val=0)`	Symétrie par rapport à x=val, y=val ou z=val (ENU ou ECEF).
`shiftTo(idx_point, new_coords)`	Translate la trace pour que le point idx_point arrive en new_coords (ENU).

5.11 Visualisation

Fonction / méthode	Rôle
`plot(sym='k-', type='LINE', af_name='', cmap=-1, append=True, ...)`	Trace la trace (matplotlib). Renvoie l'objet Axes.
`plotAsMarkers(size, frg, bkg, sym_frg, sym_bkg, ...)`	Trace les points sous forme de marqueurs.
`plotEllipses(sym, factor, af, ...)`	Trace les ellipses d'incertitude à partir d'une AF contenant une matrice de covariance 2x2.
`plotProfil(template='SPATIAL_SPEED_PROFIL', afs, ...)`	Profil de la trace.
`plotAnalyticalFeature(af_name, template='BOXPLOT', ...)`	Trace les valeurs d'une AF selon l'abscisse curviligne.
`plotFirstObs(...), plotLastObs(...)`	Marque le début (S) et la fin (E) de la trace.

5.12 Opérateurs Python

Fonction / méthode	Rôle
`t1 + t2`	Concatène les points de deux traces.
`t / n`	Découpe la trace en n parties égales ; renvoie une TrackCollection.
`t > n, t < n`	Retire les n premiers / derniers points. Entre deux traces : comparaison temporelle (t1 commence après la fin de t2, etc.).
`t >= x, t <= x`	Avec un nombre : appelle removeIdleEnds au début / à la fin. Entre deux traces : inclusion temporelle.
`t ** n`	Rééchantillonne la trace en n points (temporel, linéaire).
`t1 - t2`	Appariement des deux traces (match, DTW par défaut).
`t * x`	Entre deux traces : intersection. Avec un nombre : rééchantillonnage avec facteur.
`t % n`	Garde un point sur n, ou selon un motif booléen donné en liste.
`t1 // t2`	Rééchantillonne t1 aux instants de t2.
`t[i], t[a:b]`	Un point, ou une sous-trace.
`t['af'], t['af', i], t['expr']`	Vecteur d'une AF ; valeur d'une AF au point i ; expression algébrique.
`t['nom'] = valeur`	Crée / met à jour une AF ; avec une fonction : calcule l'AF ; avec '#DELETE' : la supprime.

6. TrackCollection (module core.track_collection)

Ensemble de traces. TrackCollection(TRACES=[]) copie superficiellement la liste : les objets Track eux-mêmes ne sont pas dupliqués.

Fonction / méthode	Rôle
`addTrack(track)`	Ajoute une trace.
`removeTrack(...), removeEmptyTrack()`	Supprime une trace ; supprime les traces vides.
`getTrack(i), getTracks()`	Une trace ; la liste des traces.
`getTrackWithTid(tid)`	Première trace ayant l'identifiant donné.
`randNTracks(N)`	Échantillon aléatoire de N traces.
`copy()`	Copie profonde (chaque trace est copiée).
`size(), len(c)`	Nombre de traces.
`length(), duration(), frequency()`	Statistiques sur l'ensemble des traces.
`summary()`	Résumé de la collection.
`toWKT()`	Chaîne WKT MULTILINESTRING (voir remarque en section 13).
`getSRID()`	Système de coordonnées (lu sur la première trace).
`toGeoCoords(), toENUCoords(), toECEFCoords()`	Conversion de toutes les traces.
`setTimeZone(), convertToTimeZone()`	Fuseau horaire de toutes les traces.
`bbox(), filterOnBBox(bbox)`	Boîte englobante ; filtre sur une boîte.
`createSpatialIndex(), exportSpatialIndex(), importSpatialIndex()`	Création, sauvegarde et chargement d'un index spatial.
`resample(), smooth(), noise(), simplify(tolerance), operate(), addAnalyticalFeature()`	Applique le traitement à chaque trace de la collection.
`segmentation(afs_input, af_output, thresholds_max)`	Marque des segments selon des seuils sur des AF.
`split_segmentation()`	Découpe les traces selon le marquage produit par segmentation.
`plot()`	Trace toutes les traces.
`c1 + c2`	Concaténation : nouvelle collection (les traces ne sont pas copiées). Les deux opérandes doivent être des TrackCollection.

Fonction / méthode	Rôle
`c[i], c[a:b]`	Une trace ; une sous-collection.

Pour la sauvegarde sur disque, aucune méthode de TrackCollection n'écrit de fichier de traces. Dans la documentation, l'export passe par TrackWriter.writeToFiles(collection, dossier, ext='csv', id_E, id_N, id_U, id_T, h, separator) : un fichier par trace.

7. Simplification (module algo.simplification)

Fonction / méthode	Rôle
`simplify(track, tolerance, mode=MODE_SIMPLIFY_DOUGLAS_PEUCKER, verbose=True)`	Fonction générique de simplification d'une trace. La signification de tolerance dépend du mode (voir ci-dessous).
`douglas_peucker(track, eps)`	Algorithme de Douglas-Peucker : réduit le nombre de points en gardant la forme ; eps = écart maximal autorisé à la ligne droite.
`visvalingam(track, eps)`	Algorithme de Visvalingam : supprime les points formant les plus petits triangles ; eps = racine de l'aire du triangle.
`squaring(track, eps)`	Redresse les angles proches de l'angle droit (moindres carrés itératifs) ; eps = seuil angulaire en radians.
`optimalSimplification(track, cost, eps, mode)`	Simplification par programmation dynamique avec une fonction de coût.
`compareWithDouglasPeuckerSimplification(track, threshold)`	Nombre de points de la trace la plus généralisée par Douglas-Peucker respectant une qualité donnée.
`averageOffsetDistance(track, threshold)`	Distance moyenne entre les points d'origine et la trace simplifiée (mesure de qualité).

Modes de simplify

Constante	Signification
`MODE_SIMPLIFY_REM_POS_DUP = 0`	Supprime les doublons de position au-delà d'un seuil de distance.
`MODE_SIMPLIFY_DOUGLAS_PEUCKER = 1`	Douglas-Peucker (défaut) : tolerance = écart maximal à la ligne droite.
`MODE_SIMPLIFY_VISVALINGAM = 2`	Visvalingam : tolerance = aire maximale du triangle de trois points consécutifs.
`MODE_SIMPLIFY_SQUARING = 3`	Squaring : tolerance = seuil sur les angles plats et droits.
`MODE_SIMPLIFY_MINIMIZE_LARGEST_DEVIATION = 4`	Minimise la plus grande déviation à la ligne droite.
`MODE_SIMPLIFY_MINIMIZE_ELONGATION_RATIO = 5`	Minimise le rapport d'élongation du rectangle englobant minimal.
`MODE_SIMPLIFY_PRECLUDE_LARGE_DEVIATION = 6`	Interdit toute déviation supérieure au seuil.
`MODE_SIMPLIFY_FREE = 7, MODE_SIMPLIFY_FREE_MAXIMIZE = 8`	Fonction de coût fournie par l'utilisateur, à minimiser ou à maximiser.

8. Comparaison, appariement et fusion (module algo.comparison)

Fonction / méthode	Rôle
`compare(track1, track2, mode=MODE_COMPARISON_POINTWISE, p=1, dim=2, verbose=False, plot=False)`	Mesure une distance (ou pseudo-distance) entre deux traces. p : exposant de la norme Lp (p = inf pour la norme infinie). dim : 1 altimétrique, 2 planimétrique, 3 en 3D, ou une fonction de distance.
`match(track1, track2, mode=MODE_MATCHING_DTW, p=1, dim=2, ...)`	Apparie deux traces (couplage n:m). Renvoie une trace de la taille de track1, avec les AF diff, pair, ex, ey, indiquant pour chaque point son ou ses homologues dans track2.
`fusion(tracks, mode, master, p, dim, represent_method, agg_method, constraint, recursive, iter_max, verbose, log, quality, cv)`	Fusion récursive d'une TrackCollection en une trajectoire médiane (approche de L. Etienne). Itère appariement et agrégation jusqu'à convergence. Renvoie une Track.
`plotMatching(matching, track2, af_name='pair', ...)`	Trace les liens d'appariement entre deux traces.

Fonction / méthode	Rôle
`printMatching(matching)`	Affiche les détails d'un appariement (outil de débogage).
`premiereComposanteHausdorff(track1, track2)`	Distance de Hausdorff dirigée de track1 vers track2.
`benchmark()`	Banc d'essai de la fusion sur des traces synthétiques.

Modes de comparaison (compare)

Constante	Signification
`MODE_COMPARISON_POINTWISE = 101`	Distance entre points de même indice (traces de même taille).
`MODE_COMPARISON_NN = 102`	Distances entre plus proches voisins.
`MODE_COMPARISON_HAUSDORFF = 103`	Distance de Hausdorff (symétrique).
`MODE_COMPARISON_AREAL = 104`	Distance aréale de McMaster.
`MODE_COMPARISON_RADIAL = 105`	Distance radiale (boucles fermées).
`MODE_COMPARISON_DTW = 106`	Distance issue du Dynamic Time Warping.
`MODE_COMPARISON_FDTW = 107`	Distance issue du DTW rapide (approché).
`MODE_COMPARISON_FRECHET = 108`	Distance de Fréchet discrète.
`MODE_COMPARISON_SYNC = 109`	Comparaison après synchronisation temporelle.
`MODE_COMPARISON_DI_HAUSDORFF = 110`	Hausdorff dirigé de A vers B.

Modes d'appariement (match et fusion)

Constante	Signification
`MODE_MATCHING_NN = 1`	Plus proche voisin.
`MODE_MATCHING_DTW = 2`	Dynamic Time Warping (norme Lp).
`MODE_MATCHING_FDTW = 3`	DTW rapide et approché.
`MODE_MATCHING_FRECHET = 4`	Appariement de Fréchet discret.

Paramètres de la fusion

Constantes	Signification
`MODE_MASTER_RANDOM / MEDIAN_LEN / MEDIAN_GEOM / MORE_DETAILED / LONGEST_MBR`	Choix de la trace de référence initiale : au hasard ; longueur médiane ; somme des distances minimale ; plus de points après simplification ; plus long rectangle englobant.
`MODE_REP_BARYCENTRE / MEDIAN_TIME / FURTHEST_OBS`	Point représentatif d'un groupe de points appariés : barycentre ; position au temps médian ; point le plus éloigné de la trace maître.
`MODE_AGG_MEDIAN / L1 / L2 / LInf`	Agrégation d'un groupe : médiane composante par composante ; médiane géométrique ; barycentre ; centre du cercle minimal.

9. Calage et map-matching (module algo.mapping)

Fonction / méthode	Rôle
`mapOnTrack(coord_or_track, track)`	Projette des positions (ou les points d'une trace) sur une trace. Renvoie les points projetés avec deux AF : dist (distance à la projection) et edge (indice du segment).
`mapOnNetwork(tracks, network, gps_noise=50, transition_cost=10, search_radius=50, debug=False, verbose=False)`	Map-matching d'une trace ou d'une collection sur un réseau, par modèle de Markov caché (méthode de Newson et Krumm). Résultats stockés en AF : hmm_inference (projection, arête, distances aux sommets), obs_noise, hmm_cost. Ne renvoie rien.
`mapOnRaster(track, raster)`	Transfère les valeurs d'un raster vers la trace : une AF par grille du raster.
`mapOn(track, reference, TP1, TP2, init, apply, N_ITER_MAX, NPTS, mode='2D', verbose)`	Transformation affine (rotation, échelle, translation) alignant une trace sur une autre, par moindres carrés sur des points de calage ; sans points de calage, par ICP (iterative closest point). Mode '3D' par décomposition en valeurs singulières. Renvoie [theta, k, tx, ty] en 2D.

10. Fonctions géométriques (module util.geometry)

La page d'API liste les fonctions suivantes ; seules certaines ont une description dans la documentation.

Fonction / méthode	Rôle
`cartesienne`	Équation cartésienne d'une droite.
`dist_point_droite`	Distance d'un point à une droite (rôle déduit du nom).
`dist_point_to_segment`	Distance d'un point à un segment (rôle déduit du nom).
`distance_to_segment(x0, y0, x1, y1, x2, y2)`	Distance entre le point (x0, y0) et le segment (x1, y1)-(x2, y2).
`projection_droite`	Projection orthogonale sur une droite.
`proj_segment`	Projection d'un point sur un segment (rôle déduit du nom).
`proj_polyligne(X, Y, x, y)`	Projection d'un point sur une polyligne. Renvoie notamment la distance minimale, les coordonnées de la projection et l'indice du segment.
`detect_side`	Détection du côté d'une trace par rapport à un point (rôle déduit du code).
`triangle_area`	Aire d'un triangle par produit vectoriel.
`aire_visval`	Aire du triangle formé par un point et ses voisins (utilisée par Visvalingam).
`isSegmentIntersects, intersection, intersects`	Intersection de segments / de géométries (rôle déduit des noms).
`inclusion`	Test d'inclusion (rôle déduit du nom).
`transform, transform_inverse`	Transformation de coordonnées et sa réciproque (rôle déduit des noms).
`azimut`	Calcul d'un azimut.
`collinear`	Test de colinéarité de trois points [x, y].
`angleBetweenThreePoints`	Angle entre trois points, calculé au point central.

11. Topology (module util.topology)

Classe de méthodes statiques pour construire une structure topologique de réseau.

Fonction / méthode	Rôle
`Topology.create_topology(input_file, srid, output_file, tbp=1e-06, verbose=False)`	Crée un réseau topologique de base. input_file : fichier texte listant des géométries ligne ou multiligne en WKT. srid : système des sommets. output_file : fichier de sortie. Ne renvoie rien.
`Topology.create_geoms_topology(collection, tbp=1e-06, verbose=False)`	Crée la topologie à partir d'une TrackCollection représentant les arêtes. tbp : tolérance de rapprochement géométrique. Renvoie une liste de tuples (edge_id, source_node_id, target_node_id, line_wkt).

12. Entrées / sorties

Fonction	Rôle
`NetworkReader.readFromFile(path, formatfile='DEFAULT', verbose=True)`	Charge un réseau depuis un fichier texte vers une structure Network. formatfile : un NetworkFormat ou le nom d'un format enregistré (ex. 'VTT', dossier tracklib/resources/). SRID par défaut : ENU. Le réseau doit fournir pour chaque arête un nœud source et cible, un identifiant, une géométrie LineString ; 'orientation' et un coût de parcours sont optionnels.
`NetworkWriter`	Classe d'écriture de réseau, listée dans la documentation mais non détaillée ici.
`TrackWriter.writeToFiles(collection, dossier, ext='csv', id_E, id_N, id_U, id_T, h, separator)`	Écrit chaque trace d'une collection dans un fichier distinct (format selon ext : CSV, GPX ou WKT). Signature relevée dans un exemple de la documentation ; le sens de h n'y est pas expliqué.
`Lecture de traces (TrackReader, TrackFormat)`	Les guides de la documentation décrivent le chargement de traces depuis des fichiers ; ces classes ne figurent pas dans la section API et ne sont pas détaillées ici.

13. Points d'attention relevés dans le code

Ces remarques viennent de la lecture du code source, sans exécution. À vérifier avant de vous y fier.

• Track.noise() renvoie une trace ; il faut réaffecter le résultat (t = t.noise(...)). Le paramètre n est accepté mais n'est pas transmis (la fonction interne reçoit toujours n=1). TrackCollection.noise() remplace chaque élément de sa liste par le résultat.

• Track.smooth() fait self = filter_seq(...) : cette réaffectation ne concerne que la variable locale, donc la trace appelante semble ne pas être modifiée. Vérifier le comportement avant d'utiliser cette méthode.

• Track.scale(h) et Track.translate() agissent en place, ne renvoient rien et exigent des coordonnées ENU. ENUCoords.scale(h) multiplie E et N mais pas U ; pour agrandir aussi l'altitude, utiliser Track.scale3D.

• Plusieurs méthodes (toENUCoords, scale, rotate, resample, toProjCoords...) appellent exit() en cas d'erreur de système de coordonnées : cela arrête tout le programme au lieu de lever une exception.

• Track.resample() remet à zéro le dictionnaire des analytical features : tout ce qui a été calculé avant est perdu.

• Track.removeObsDup() est vide dans la version lue (renvoie None). Pour supprimer des doublons, utiliser cleanDuplicates() ou removePosDup().

• Track.removeTpsDup() renvoie une nouvelle trace, alors que removePosDup() modifie la trace en place.

• Les contrôles de type de Obs (distanceTo, etc.) lèvent CoordTypeError, mais l'import des exceptions est commenté dans core.obs : l'erreur réellement levée pourrait être une NameError.

• TrackCollection.toWKT() ne place pas de virgule entre les lignes du MULTILINESTRING, ce qui peut être rejeté par un analyseur WKT strict. Track.toWKT() (une seule trace) produit un LINESTRING correct.

• Les signes de __sub__ ne sont pas homogènes : ENUCoords renvoie self - p, ECEFCoords renvoie p - self. Sans conséquence pour les distances, mais à garder en tête pour des différences vectorielles.

• La classe se nomme Bbox dans le code (pas BBox). La page d'API la présente comme 'Bounding Box'.

14. Ce que ce document ne couvre pas

La documentation publique expose seulement une partie de la bibliothèque dans sa section API. Les modules suivants sont mentionnés dans le README ou utilisés dans le code, mais je n'ai pas pu en lire le code ni l'API, donc ils ne sont pas détaillés ici :

• Génération de données synthétiques (tkl.generate, module synthetics) et générateurs aléatoires (seed).

• Noyaux (GaussianKernel, ExponentialKernel, DiracKernel, TriangularKernel, SincKernel).

• Opérateurs (classe Operator et ses sous-classes) utilisés par operate().

• Algorithmes de filtrage de Kalman, de segmentation, de classification, de rééchantillonnage, d'estimation cinématique (Cinematics), HMM.

• Classes Network, Raster et AFMap ; lecture de fichiers de traces (TrackReader, TrackFormat) ; visualisation (MatplotlibVisitor).

• Fonctions de util (Polygon, boundingShape, centerOfPoints, etc.) autres que celles listées en section 10.

Pour la liste exhaustive, la source de référence reste le dossier tracklib/ du dépôt GitHub, ou dans Python : help(tkl.NomDeLaFonction) et inspect.getsource().